

OnTime

For Real Estate Owners and Property
Managers



By: Jiovanni Bycinte, Kenneth Villard,
Alon Bauman

Functional Requirements

Create Maintenance Tasks

- Log maintenance requests
- Set deadlines and priorities
- Organize tasks by property/location

Assign Vendor

- Assign tasks to specific vendors
- Vendors receive task notifications
- Ensures clear responsibility

Track Task Status

- Monitor progress in real time
- Identify overdue tasks
- Update completion status

Generate Vendor Reliability Reports

- Track response & completion times
- Record missed deadlines
- Evaluate vendor reliability over time

Non-functional Requirements



Usability

- Simple, intuitive interface
 - Quick task creation & assignment
 - Minimal training required
-

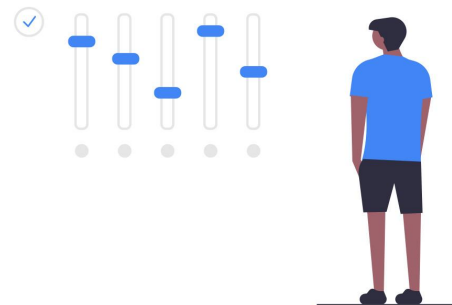
Performance

- Real-time updates
 - Fast response times
 - Supports multiple users
-

Security

- Secure authentication
 - Role-based access control
 - Encrypted data transmission
-

System Constraints



1 Tool Constraints

- Development uses GitHub and VS Code
- Cloud tools required for hosting & deployment

Impact: ensures collaboration, version control, and efficient development

2 Language Constraints

- Built using languages like JavaScript, Python. Must support maintainability and scalability

Impact: enables long-term support and developer accessibility

3 Platform Constraints

- Mobile-responsive interface needed

Impact: ensures accessibility and flexible access from any device

Network Constraints



Google Cloud

1 Reliable Hosting

The service must run on a trusted cloud platform like AWS or Google Cloud

2 Always Online

The system needs high uptime so it remains available to users without unexpected interruptions

3 Low Maintenance

The cloud setup should be hands-off so our team doesn't have to worry about daily backend management.

- Because the servers require low maintenance, our team can focus entirely on building and improving features instead of wasting time fixing server crashes.
- Since the system is always online, users will have a reliable, frustration-free experience, which builds their trust in the app

Hardware Constraints



- 1 **Standard Devices** Users simply need an everyday device, like a smartphone, tablet, or computer, to access the app
- 2 **No Special Equipment** There is absolutely no need to buy extra or specialized hardware to make the system work
- 3 **Highly Accessible** Because it runs on devices people already own, OnTime is instantly accessible to almost anyone.

Planning & Transition Constraints

Transition & Support

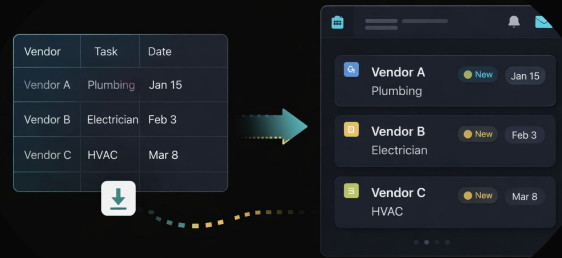
- User onboarding guidance
- Data migration (existing tasks/vendors)
- In-platform support & updates

Impact: ensures smooth user platform adoption

Budget & Schedule

- 10–12 week timeline
- No external developers
- Low-cost / free-tier cloud hosting

Impact: enables successful data migration without disruption



Impact: Ensures project feasibility

Miscellaneous Constraints



1 Data Protection Compliance

The platform must comply with applicable data protection standards.

Impact: ensures secure handling of user and vendor data

2 Ethical Practice

Software will be created for ethical practices only.

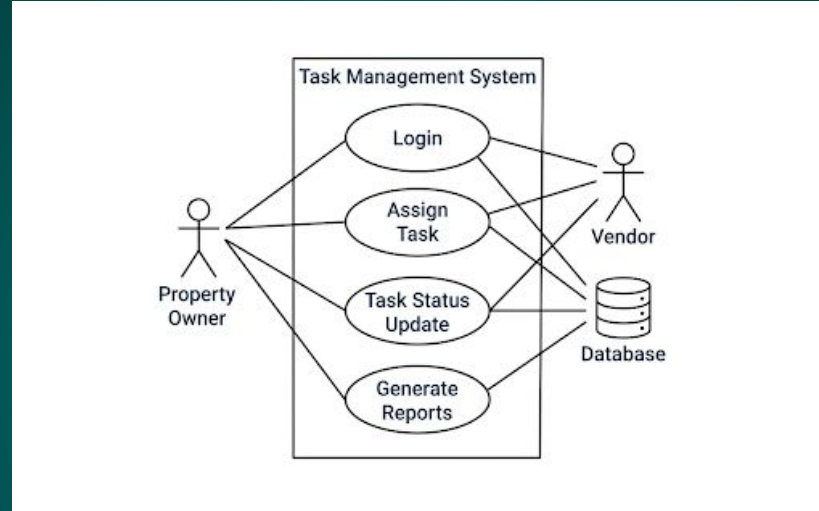
Impact: promotes responsible and unbiased system usage

3 Fair Vendor Evaluation

Vendor performance metrics must be based solely on recorded system data.

Impact: prevents bias and ensures objective vendor assessment

Use case diagrams



Main system Functionality

- **Task Creation:** Shows how a Property Owner creates a task and assigns it to a Vendor.
- **Task Update:** Allows for Property Owner to edit the task and vendor to display progress
- **Generate Report:** Vendor generates report to send back to the Property Owner.

User and vendor login

- **Account Basics:** Displays the core account actions: registering, logging in, logging out, and resetting passwords.

Evolutionary Functional Requirements

Automated Vendor Rating Algorithm

- Automatically calculate vendor reliability score
 - Analyze completion time and response time
 - Detect repeated missed deadlines
-

SMS / Push Notifications

- Send real-time task assignment alerts
 - Notify vendors of deadline changes
 - Trigger overdue task reminders
-



Evolutionary Non-Functional Requirements

Scalability:

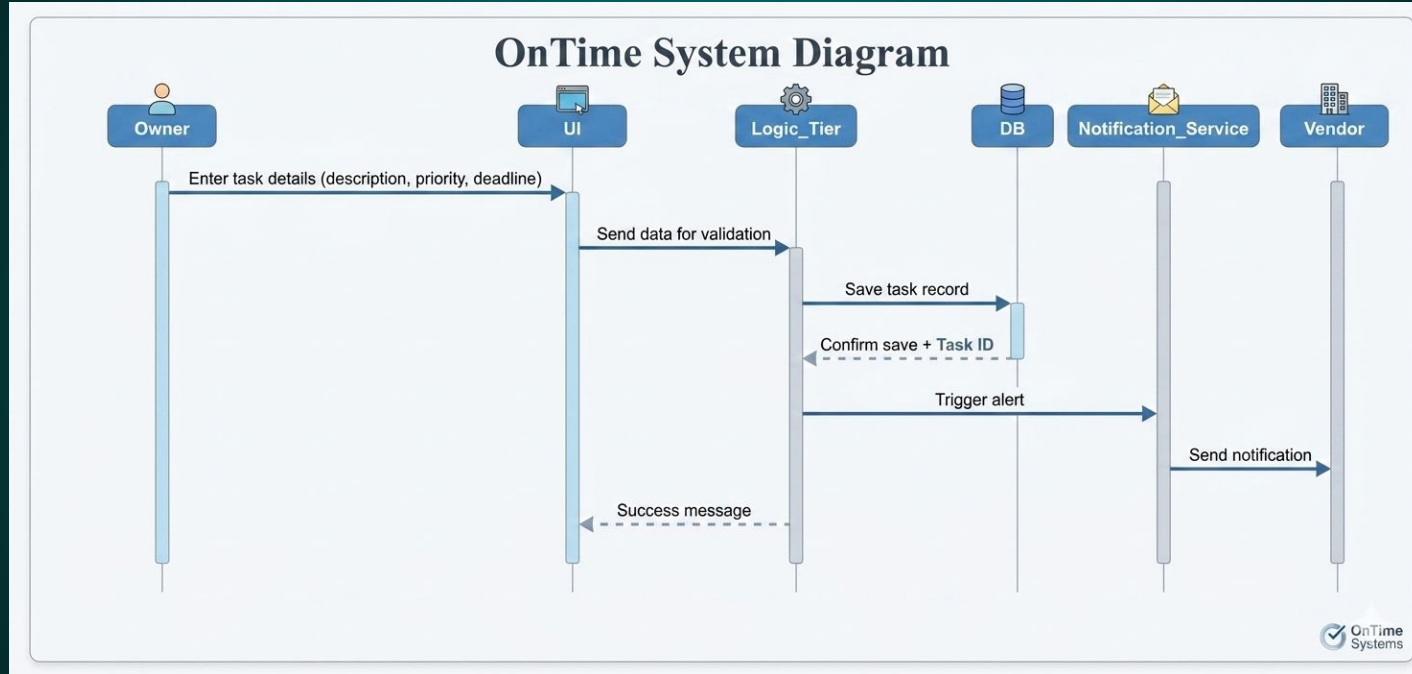
- System must support 10,000+ concurrent users without performance degradation

Impact: enables long-term system growth beyond initial deployment

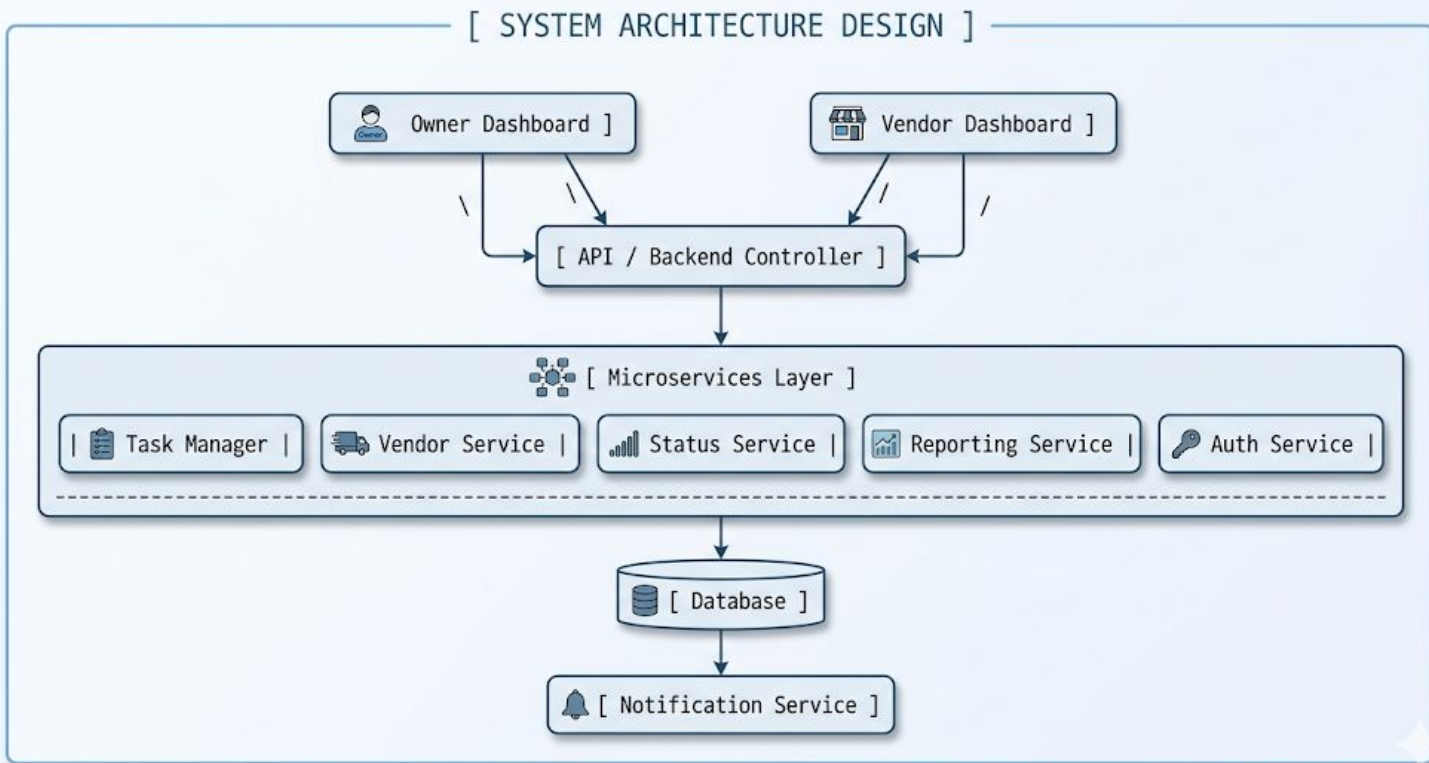


System Model

TASK STATUS UPDATE



System Architecture Design



Design Patterns

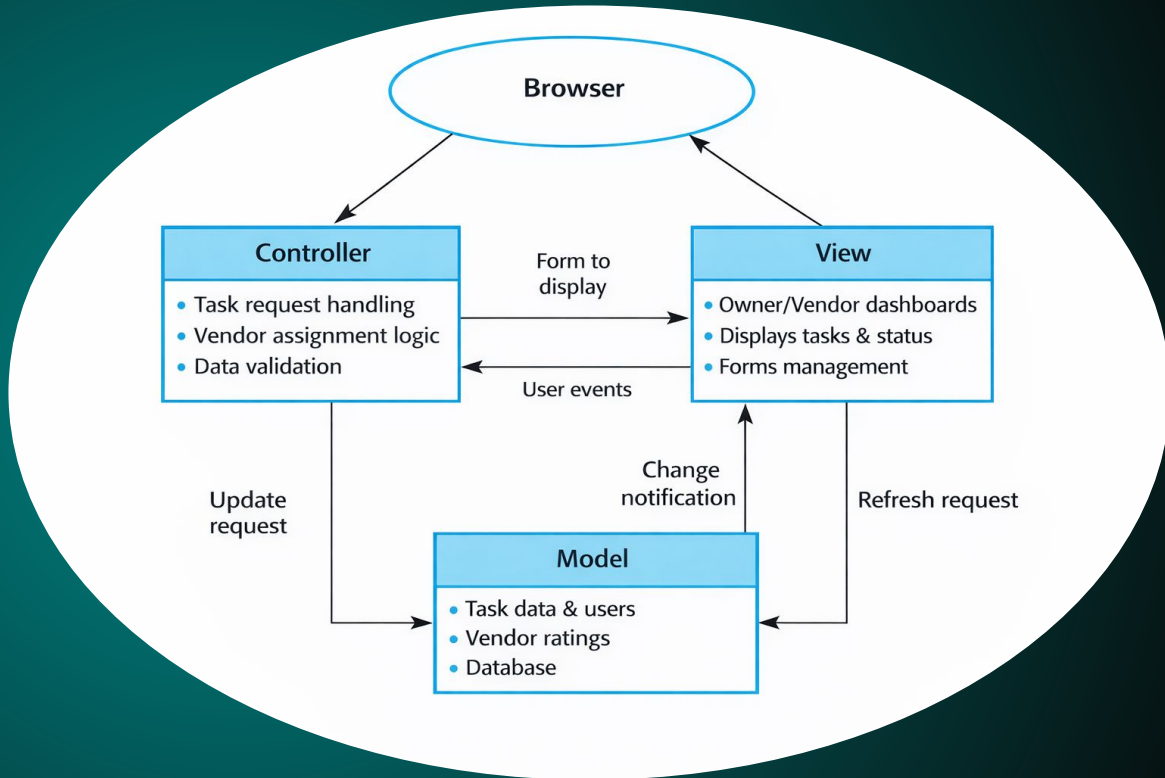
Model-View-Controller (MVC)

Model: Task data, users, vendor ratings

View: Owner/Vendor dashboards (UI)

Controller: Processes user actions (create tasks, update status)

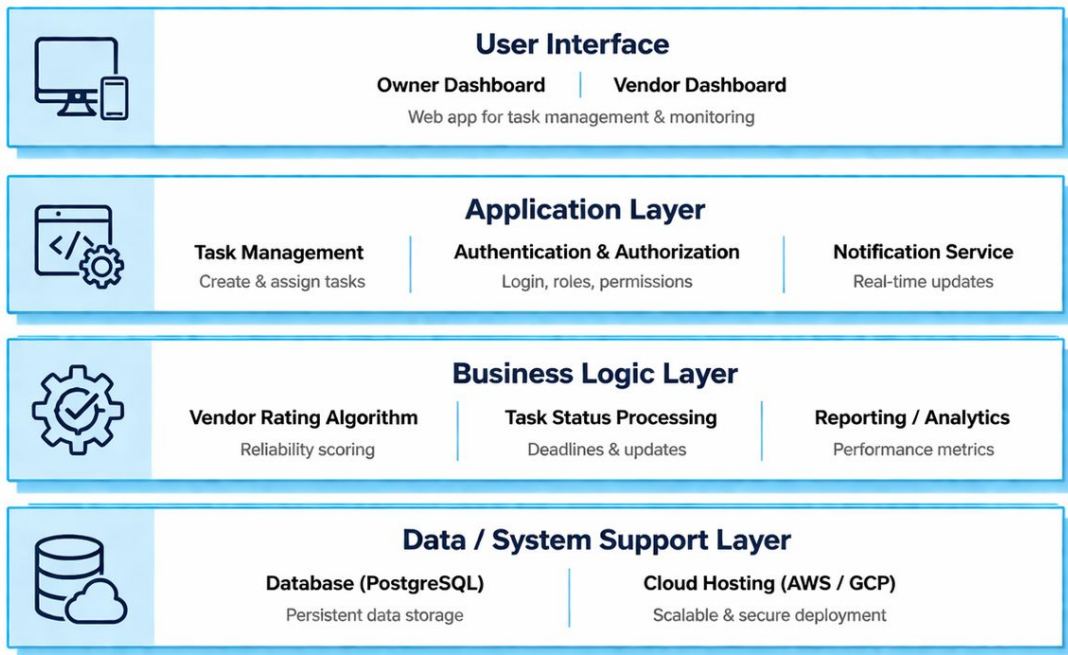
Separates UI, logic, and data → improves maintainability and scalability



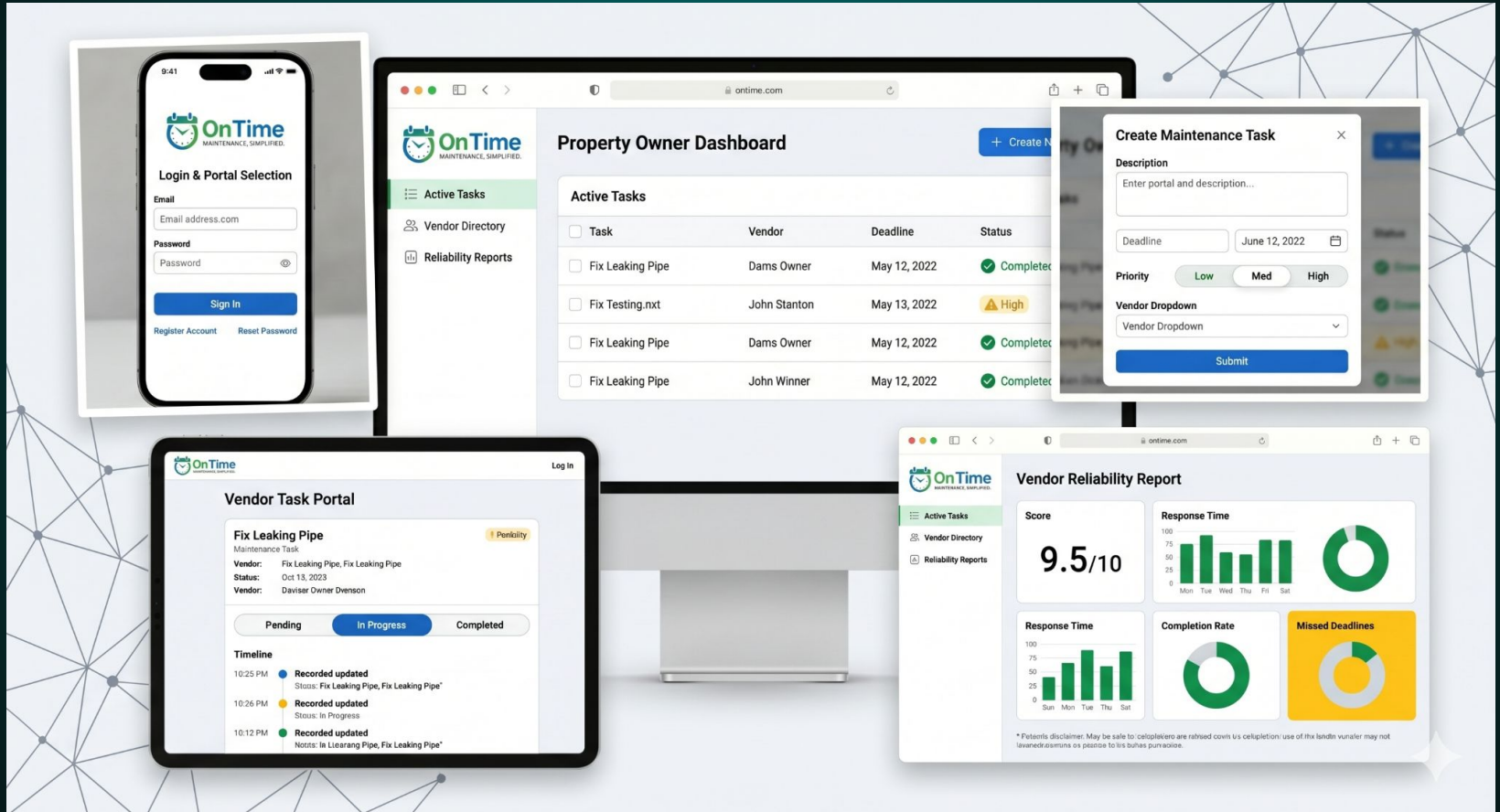
Design Patterns

Design Patterns

Layered Architecture (MVC)



Mockup Interface



Conclusion

On Time is designed with:

- Structured planning
- Financial feasibility
- Ethical requirements
- Scalable architecture

Result: A maintainable and expandable platform that streamlines property manager–vendor interactions.



Resources

- <https://docs.cloud.google.com/architecture/framework>
- <https://www.cloudflare.com/en-gb/learning/security/threats/owasp-top-10/>
- <https://cpl.thalesgroup.com/software-monetization/four-types-of-software-maintenance>
- <https://sequencediagram.org/>

Questions?



Vote for TEAM 3

DynaFlip.com/p1103

